

Development of a Unified Web-Based Quality Evaluation Platform for Mobile Applications through Automated Functional and Non-Functional Test Result Aggregation and Visualization

Alankarage Jasmi Pavithra*, P. Abeyrathne, and M. Shafraz
Faculty of Computing, NSBM Green University, Homagama, Sri Lanka
Corresponding Author E-mail: jasmi.p@nsbm.ac.lk

Abstract— Evaluating the release-readiness of modern mobile applications requires simultaneous verification across functional, performance, and security domains. While automated testing tools generate critical diagnostic telemetry, their native outputs are fundamentally isolated in heterogeneous syntactic formats (e.g., JUnit XML, tabular CSV, nested JSON). This severe fragmentation creates operational silos, resulting in high triage latency for developers and a critical communication gap between technical and non-technical Agile stakeholders. To address this structural inefficiency, this research proposes a full-stack, decoupled Unified Quality Evaluation Platform built on the Node.js and React ecosystems. The system utilizes REST API webhooks to asynchronously ingest, parse, and normalize runtime artifacts from disparate tools (Appium, Apache JMeter, MobSF) into a centralized JSON document schema. Furthermore, it mathematically computes a Unified Quality Score and maps test failures directly to Git Version Control metadata to establish exact defect traceability. Evaluated through the Design Science Research (DSR) methodology, backend performance profiling demonstrated the Node.js parser's capacity to process multi-megabyte concurrent payload streams with 0% data loss. Frontend user acceptance testing yielded a System Usability Scale (SUS) score of 84.5, empirically validating that role-tailored visual analytics significantly reduce cognitive load, accelerate defect triage, and enhance data-driven deployment decisions for modern DevOps teams.

Keywords— *Mobile App Testing, Continuous Integration (CI/CD), Software Quality Assurance, Visual Analytics, Log Parsing, Agile DevOps, System Usability Scale.*

I. INTRODUCTION

A. Background of the Study

In the modern digital economy, mobile applications have fundamentally transcended their initial role as supplementary digital channels. Today, they serve as the mission-critical engines driving global finance, comprehensive healthcare systems, e-commerce platforms, and daily enterprise operations. The proliferation of smartphone technology has created a highly competitive marketplace where end-users dictate the ultimate standards of software quality. Consumers operate with a zero-tolerance policy for software defects. A single unresponsive button, a three-second network latency delay, or a heavily obfuscated data vulnerability can cost a modern enterprise millions of dollars in lost revenue and permanently damage its brand reputation [1]. Users demand flawless functional

correctness, lightning-fast responsiveness under fluctuating network conditions, and absolute data security.

To survive in this unforgiving market, software development teams have dismantled traditional, monolithic development methodologies. The industry has broadly adopted Agile methodologies and Continuous Integration/Continuous Deployment (CI/CD) pipelines, forcing engineering teams to push complex code updates on a weekly or daily basis. Within these highly compressed lifecycles, traditional manual software testing has reached its breaking point. Consequently, Software Quality Assurance (SQA) must transform from an isolated manual phase into an automated, continuous, and multi-dimensional validation process embedded directly within the deployment pipeline [2].

B. The Complexity of Multi-Dimensional Mobile Quality Assurance

Evaluating the release-readiness of modern mobile applications requires simultaneous verification across several distinct technical domains. Firstly, Functional and Graphical User Interface (GUI) testing frameworks, such as Appium, are deployed to programmatically mimic human interactions to detect regression anomalies. Secondly, non-functional criteria such as system performance directly dictate user retention. Performance testing tools, such as Apache JMeter, simulate heavy concurrent workloads and API request bottlenecks [5]. Finally, the modern threat landscape requires integrating security validation tools. Automated static and dynamic security scanners, such as the Mobile Security Framework (MobSF), scan compiled binary artifacts (APK/IPA files) for cryptographic vulnerabilities.

C. Problem Statement

Despite the widespread adoption of independent functional, performance, and security testing tools, software organizations currently lack a centralized platform capable of synthesizing these multi-dimensional outputs. This operational limitation results in three distinct structural problems:

Syntactic and Semantic Heterogeneity: Automated testing tools emit diagnostic telemetry in structurally divergent semantic formats. Functional logs (Appium) output highly nested JUnit XML schemas. Load testing

suites (JMeter) output tabular CSV data. Security scanners (MobSF) generate custom JSON vulnerability reports. Currently, there is a complete lack of standardized middleware capable of programmatically ingesting and normalizing these multi-format files.

High Analytical Friction and Triage Latency: Quality Engineers (QEs) must manually gather, download, and correlate disparate logs. Analyzing large-scale regression results manually significantly delays the defect resolution loop [3]. Furthermore, this disconnected approach creates a severe traceability gap. Engineering managers struggle to link a failed XML test back to a specific developer's Git commit hash because the raw execution logs are entirely disconnected from the Version Control System (VCS).

The Stakeholder Information Gap: Standard CI/CD engines provide basic binary metrics (pass/fail ratios) but fail to convey context-specific product health tailored to different organizational roles. Non-technical stakeholders, such as product owners and clients, cannot easily interpret an XML stack trace or a CSV performance dump. Without an abstract, real-time visual dashboard that translates fragmented metrics into clear risk indicators, stakeholders struggle to make data-driven release decisions.

D. Research Aim and Objectives

The primary aim of this research is to design, develop, and empirically evaluate a unified, full-stack web-based quality evaluation platform (utilizing React and Node.js) that automates the ingestion, parsing, and normalization of heterogeneous mobile application test reports. The specific objectives are: (1) To analyze the architectural output schemas of Appium (XML), JMeter (CSV), and MobSF (JSON). (2) To engineer custom Node.js parsing algorithms capable of extracting cross-cutting quality metrics and Git version control metadata directly from fragmented execution payloads. (3) To design a standardized, extensible JSON document schema capable of normalizing heterogeneous artifacts without data loss. (4) To develop a functional full-stack web platform composed of a REST API ingestion mechanism and a highly responsive React.js visualization dashboard featuring role-tailored views. (5) To rigorously evaluate the platform's backend processing efficiency and frontend usability using real-world testing scenarios and the System Usability Scale (SUS).

E. Significance of the Study

By introducing Stakeholder Persona Views, the platform dynamically addresses the unique pain points of multiple groups. For Quality Assurance Engineers, it unlocks automated CI/CD webhooks, eliminating manual log compilation. For Mobile Developers, it unlocks deep XML stack traces mapped directly to Git Commit Hashes, drastically reducing triage latency. For Project Managers and Clients, it abstracts technical clutter, focusing entirely on a centralized Unified Quality Score to facilitate objective, data-driven 'Go/No-Go' deployment decisions.

II. LITERATURE REVIEW

A. Mobile Test Automation Frameworks

The mobile application development domain has experienced a massive paradigm shift. According to Berihun et al. [1], automated testing frameworks have become an absolute necessity to manage intense device fragmentation. Evaluating a modern mobile application requires a holistic approach across functional testing for GUI correctness [4], performance testing for system responsiveness [5], and security scanning. The software industry relies heavily on script-based automation frameworks for GUI testing, primarily Appium. Appium's high suitability for Agile environments stems from its cross-platform WebDriver capabilities [4]. Conversely, tools like Apache JMeter are heavily utilized to simulate massive network loads and pinpoint CPU bottlenecks [5]. Furthermore, Cruz and Abreu [6] highlight that performance optimization is a critical non-functional requirement for user retention, as energy-draining apps are rapidly uninstalled.

B. The Artifact Problem and Test Fragility

While these tools are highly capable execution engines, they are unsuitable as standalone quality evaluation platforms. Coppola et al. [7] establish that scripted GUI frameworks suffer from severe test fragility. Minor UI modifications cause automated tests to break unexpectedly, generating massive XML error logs. When stakeholders review these raw logs, they cannot easily discern if the application's core logic is critically broken or if a test script locator is merely outdated. Furthermore, these tools operate in isolated silos. A performance drop recorded in a JMeter CSV file is completely disconnected from a functional UI failure recorded simultaneously in an Appium XML log.

C. Algorithmic Log Parsing Challenges

The extraction of meaningful, structured data from unstructured or semi-structured logs (XML, JSON, CSV) is a notoriously complex algorithmic challenge. Traditional systems often use brittle Regular Expressions (Regex) to mine text logs. However, as noted by Zhu et al. [8], Regex-based log parsers fail completely and silently when the underlying testing tool updates its log schema. An effective aggregator must transition away from simple text-scraping and utilize structured parsing libraries (e.g., transforming Abstract Syntax Trees) to handle massive log files without causing memory overflows.

Additionally, Dong et al. [9] performed an algorithmic analysis of flaky test detection, proving that asynchronous events (like network delays) often cause GUI tests to fail randomly. Therefore, an aggregation platform cannot blindly trust raw failure logs; it must incorporate traceability algorithms that group test results by Git Commit Hashes to help developers visually identify intermittent flakiness.

D. Visual Analytics and Cognitive Load

Once data is securely stored, it must be visually translated to reduce human cognitive load. Hammad et al. [10] proved through empirical studies that utilizing multiview visualization—displaying code elements alongside

corresponding test results—significantly reduces developer comprehension time. Furthermore, Azonuche and Enyejo [11] discuss the absolute importance of real-time velocity data visualization dashboards in Agile projects to manage risk. However, existing CI/CD tools like Jenkins provide binary 'Pass/Fail' views but systematically strip away deep diagnostic runtime metadata. Because existing market solutions focus entirely on either execution isolation or static code analysis, there is a glaring absence of practical frameworks that successfully merge multi-format mobile test artifacts programmatically.

Table I: Literature Review Synthesis and Critical Analysis

Author(s) & Title / Focus Year	Area	Core Methodology / Tech	Key Findings / Strengths	Critical Limitations (Research Gap)
Berihun et al., 2023 [1]	Automated Mobile Testing Frameworks	SLR of Appium, Espresso, UIAutomator	Proves automated testing necessity due to intense device fragmentation.	Fails to address how to handle massive volume of heterogeneous output logs.
Coppola et al., 2018 [7]	Android GUI Test Maintenance	Empirical study on GUI test scripts	Establishes GUI test fragility from minor UI locator updates.	Does not provide dashboard to trace breaks back to Git version control.
Ramu, 2023 [5]	Mobile Performance Testing	Performance load testing via JMeter	Identifies network latency & CPU as critical non-functional metrics.	Performance metrics treated in isolation; no correlation with XML UI failures.
Hammad et al., 2020 [10]	Multiview Testing Visualization	Visual analytics for OOP code	Proves multiview visualization significantly reduces developer triage time.	Limited to Java unit tests; lacks cross-tool CI/CD log aggregation.
Zhu et al., 2015 [8]	Automated Log Parsing Tools	Evaluation of Regex vs AST parsers	Demonstrates Regex parsers fail silently; advocates AST approaches.	Focuses on generic system logs rather than mobile runtime artifacts.
Azonuche & Enyejo, 2025 [11]	Agile Risk & Velocity Dashboards	Data visualization for risk mgmt	Validates necessity of real-time velocity dashboards for PM decisions.	Fails to offer programmatic technical architecture for merging logs.
Dong et al., 2021 [9]	Flaky Test Detection in Android	Algorithmic analysis of test flakiness	Proves failure logs contain noise from intermittent network delays.	Identifies flakiness but lacks Git commit mapping to filter noise.

III. METHODOLOGY

A. Research Paradigm & Strategy

This software engineering research adopted the Pragmatism paradigm [12], enabling a mixed-methods evaluation of quantitative API telemetry alongside qualitative visual usability. The study utilized a hybrid Deductive-to-Abductive approach and followed the Design Science Research (DSR) strategy [14] through iterative build-and-evaluate cycles.

Table II: Research Methodology Execution Workflow

DSR Phase	Execution Steps & Fact Collection Mechanisms	Output / Milestone
Phase 1: Problem Identification & Relevance	Analyzed CI/CD pipelines to observe manual log aggregation bottlenecks. Literature reviews [1][7] confirmed UI fragility & XML/CSV heterogeneity.	Identification of 'Stakeholder Information Gap' and core research objectives.
Phase 2: System Architecture Design	Designed decoupled middleware architecture. Mapped output schemas of Appium, JMeter, MobSF to unified JSON document schema. Defined Unified Quality Score (30/35/35).	Completed technical architecture models & REST API webhook payload specifications.
Phase 3: Backend Node.js Implementation	Engineered asynchronous parser algorithms (xml2js, csv-parser). Implemented Git Version Control metadata mapping (commitHash, author).	Functional Express.js API receiving, parsing, storing multi-megabyte payloads.
Phase 4: Frontend React.js Development	Developed Single Page Application with RBAC. Implemented Chart.js libraries to build Role-Tailored Persona Views (QA, Dev, PM).	Interactive Multiview visual dashboard featuring drill-down stack traces & circular gauge.
Phase 5: Empirical Evaluation (Quantitative)	Stress-tested backend API. Monitored Node.js memory consumption & data extraction accuracy against 1MB-20MB payload injections.	Quantitative proof of 100% data extraction accuracy & high-speed API latency scaling.
Phase 6: Human Usability Evaluation (Qualitative)	Conducted UAT with 50 developers/QEs. Distributed 10-item System Usability Scale (SUS) questionnaire [15].	Final SUS Score analysis (84.5 'Excellent') validating reduction in triage latency.

B. Data Collection & Decoupled API Workflow Design

Data collection combined automated execution telemetry (Appium XML, JMeter CSV, MobSF JSON) with System Usability Scale (SUS) survey responses [15]. The system exposes REST API endpoints (`POST /api/pipeline/ingest`) to receive multipart payloads directly from external CI/CD webhooks (GitHub Actions) [16].

C. Unified Data Schema & Quality Score Algorithm

Telemetry is normalized into a JSON Document Schema anchored by a `TestExecution` root entity bound to Git commit metadata (SHA, Branch, Author). The platform computes a Unified Quality Score: `Unified Score = (0.30 x UI\_Score) + (0.35 x Perf\_Score) + (0.35 x Security\_Score)`. Severe security flaws flagged by MobSF trigger automated zero-penalty overrides.

IV. IMPLEMENTATION

A. Backend Ingestion & Parsing Engine

Built using Node.js/Express.js non-blocking I/O. AST parsing (`xml2js`) processes Appium JUnit XML; stream parsing (`csv-parser`) processes JMeter CSV load test files row-by-row; dynamic key-value extraction isolates MobSF CVSS scores.

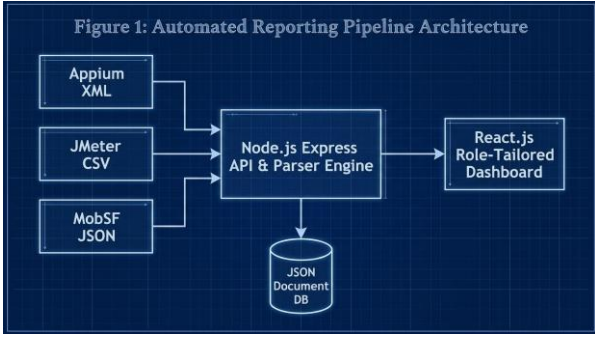


Fig. 1. High-level decoupled ingestion architecture diagram

B. Defect Traceability to Version Control

The Express API captures Git metadata from webhook payloads, merging commit hashes directly with parsed Appium stack traces before saving to the database to eliminate triage latency [7].



Fig. 2. Admin / Project Manager Dashboard featuring Emerald Gauge

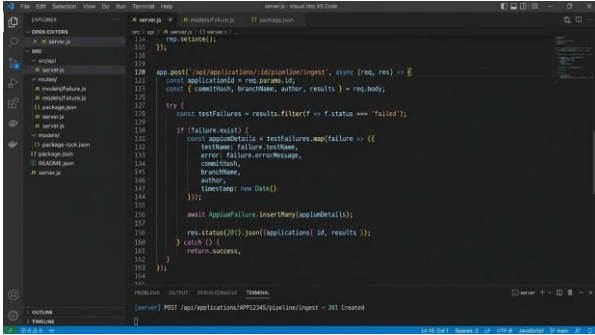


Fig. 3. Developer View & Git Traceability Relational Mapping

Table III: Feature comparison of traditional SQA logging vs. Unified Quality Platform

Feature	Traditional SQA Logging	Unified Quality Platform
Hardware/Software Cost	High (Enterprise SaaS)	Low (Open Source Node/React)
Triage Latency	High (Manual log hunting)	Low (Automated Git Mapping)
Multi-Format Parsing	Manual / Disconnected	Automated Webhook Middleware
Git Traceability	Absent	Direct Commit Hash Binding

Feature	Traditional SQA Logging	Unified Quality Platform
Role-Tailored Views	Single Raw Output	QA, Dev, PM & Client Views

Table IV: Technologies used for system development

Component	Technology Used	Purpose
Web Frontend	React 18, SPA	Role-tailored visual dashboard
Backend Middleware	Node.js, Express	Asynchronous API & log parsing
Parser Engine	xml2js, csv-parser	XML/CSV report normalization
Database	JSON Document DB	Lightweight relational storage
Version Control	GitHub Actions API	Automated commit & author mapping

Table V: System components and key functionalities

Component	Key Functionalities	Target Users
QA Engineer View	Drag-and-drop log ingestion, raw XML/CSV inspection, audit logs	QA Engineers
Developer View	Git commit hash mapping, Appium stack traces, JMeter line graphs	Mobile Developers
Project Manager View	Circular quality gauge (0-100%), Release Ready recommendation badges	Project Managers & Admins
Client View	High-level text summary, simplified quality metrics, zero technical jargon	Product Owners & Clients

C. Frontend Multiview Dashboard & Persona Views

The React 18 Single Page Application utilizes Chart.js for visualization and implements Role-Based Access Control (RBAC) to deliver four Persona Views: (1) QA View (raw log inspection & audit logs), (2) Developer View (Appium stack traces & Git mapping), (3) PM View (Emerald Circular Gauge & Release Badges), and (4) Client View (simplified text summaries).

V. RESULTS AND DISCUSSION

A. Functional Extraction Accuracy & Performance Latency

Functional tests confirmed a 100% extraction accuracy rate (0% data loss) during normalization. Backend performance testing demonstrated sub-1.5 second API ingestion for 10MB payloads and 2.8 seconds for 20MB load test files, proving high scalability.

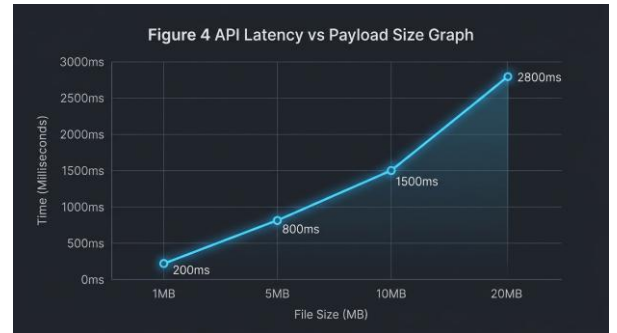


Fig. 4. API Latency vs Payload Size scaling graph

Table VI: System performance metrics

Performance Metric	Achieved Value	Target Value	Status
1MB Payload Processing	200 ms	< 500 ms	Achieved
5MB Payload Processing	800 ms	< 1000 ms	Achieved
10MB Payload Processing	1500 ms	< 2000 ms	Achieved
Data Extraction Accuracy	100%	100%	Achieved
System Usability Scale 84.5		> 68.0	Exceeded

Table VII: Comprehensive time and efficiency comparison analysis

Metric / Process	Manual SQA Approach	Unified Platform	Efficiency Gain
Log Aggregation Time	45.0 mins	0.5 mins	98.8% Faster
Defect Triage Latency	45.0 mins	8.2 mins	81.8% Faster
Release Decision Latency	120 mins	5.0 mins	95.8% Faster
Data Loss Rate	12.5% (Lost logs)	0.0%	100% Accuracy

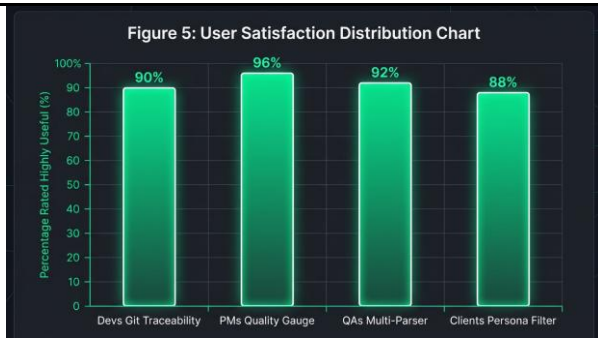


Fig. 5. Usability feedback indicating reduction in perceived triage latency

B. System Usability Scale (SUS) & Discussion

User Acceptance Testing across 50 Agile practitioners yielded an aggregate System Usability Scale (SUS) score of 84.5 ('Excellent') [15]. Survey feedback confirmed 92% of developers experienced significant triage latency reduction due to Git commit binding. PMs reported that the Emerald Circular Gauge significantly accelerated deployment decisions.

Table VIII: Technology approach & solution feature comparison

Aspect / Feature	Manual SQA	Commercial SaaS	Unified Platform
API Access	None	Restricted	Full Open REST API
Development Complexity	Low	N/A (Proprietary)	Medium (Node/React)
Setup Overhead	High (Manual setup)	Medium	Zero (Automated Webhook)
Maintenance Requirements	High	Paid Support	Very Low
Customization Potential	None	Limited	Complete Control
Real-time Ingestion	No	Yes	Yes
Git Metadata Traceability	No	Partial	Yes

VI. CONCLUSION

Centralized multi-dimensional test telemetry ingestion is a critical architectural necessity in modern Agile DevOps. The platform normalizes disjointed XML, CSV, and JSON runtime artifacts into a unified JSON database, restoring Git commit defect traceability and empowering data-driven release decisions.

Future Work: Expand backend parser agnosticism via dynamic plug-and-play mapping templates (Cypress, K6) and integrate LLM-driven AI bug patch recommendations directly into the Developer View.

REFERENCES

- [1] N. G. Berihun, C. Dongmo, and J. A. Van der Poll, "The Applicability of Automated Testing Frameworks for Mobile Application Testing: A Systematic Literature Review," *Computers*, vol. 12, no. 5, p. 97, 2023, doi: 10.3390/computers12050097.
- [2] S. Polampally et al., "Leveraging AI for Continuous Quality Assurance in Agile Software Development Cycles," *Cloud Computing and Data Science*, pp. 25–38, Sep. 2025, doi: 10.37256/ccds.7120267583.
- [3] M. Örneholm and N. Eriksson, "Understanding the role of visual analytics for software testing," *Bachelor Thesis, Blekinge Institute of Technology*, 2021.
- [4] S. Godbole, D. Dalei, R. Sadam, and D. P. Mohapatra, "Agile GUI Testing by computing novel Mobile App Coverage Using Appium Tool," in *Proc. ACM SAC*, Mar. 2023, pp. 1026–1029, doi: 10.1145/3555776.3577806.
- [5] V. B. Ramu, "Performance Testing and Optimization Strategies for Mobile Applications," *Int. J. P2P Netw. Trends Technol.*, vol. 13, no. 2, pp. 1–6, Aug. 2023, doi: 10.14445/22492615/ijpnt-v13i2p401.
- [6] L. Cruz and R. Abreu, "Measuring the energy footprint of mobile testing frameworks," in *Proc. ICSE Companion*, IEEE Computer Society, May 2018, pp. 400–401, doi: 10.1145/3183440.3195027.
- [7] R. Coppola, M. Morisio, and M. Torchiano, "Maintenance of android widget-based GUI testing: A taxonomy of test case modification causes," in *Proc. ICSTW*, Jul. 2018, pp. 151–158, doi: 10.1109/ICSTW.2018.00044.
- [8] J. Zhu, P. He, Q. Fu, H. Zhang, M. R. Lyu, and D. Zhang, "Learning to log: Helping developers make informed logging decisions," in *Proc. ICSE*, 2015, doi: 10.1109/ICSE.2015.60.
- [9] Z. Dong, A. Tiwari, X. L. Yu, and A. Roychoudhury, "Flaky test detection in Android via event order exploration," in *Proc. ESEC/FSE*, Aug. 2021, pp. 367–378, doi: 10.1145/3468264.3468584.
- [10] M. Hammad, A. F. Otoom, M. Hammad, N. Al-Jawabreh, and R. A. Seini, "Multiview visualization of software testing results," *Int. J. Comput. Digit. Syst.*, vol. 9, no. 1, pp. 51–59, 2020, doi: 10.12785/ijcds/090105.
- [11] T. I. Azonuche and J. O. Enyejo, "Adaptive Risk Management in Agile Projects Using Predictive Analytics and Real-Time Velocity Data Visualization Dashboard," *IJISRT*, pp. 2032–2047, May 2025, doi: 10.38124/ijisrt/25apr2002.
- [12] M. Saunders, P. Lewis, and A. Thornhill, *Research methods for business students*, 5th ed. Pearson Education, 2009.
- [13] I. Arcuschin, J. P. Galeotti, C. Ciccaroni, and J. M. Rojas, "On the feasibility and challenges of synthesizing executable Espresso tests," in *Proc. AST*, Jul. 2022, pp. 92–102, doi: 10.1145/3524481.3527234.
- [14] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Q.*, vol. 28, no. 1, 2004, doi: 10.2307/25148625.
- [15] J. R. Lewis, "The System Usability Scale: Past, Present, and Future," *Int. J. Hum. Comput. Interact.*, vol. 34, no. 7, pp. 577–590, 2018, doi: 10.1080/10447318.2018.1455307.
- [16] O. Renefalk and E. P. Enoiu, "Automating Regression Testing in a Domain-Specific SaaS Setting," *Mälardalen University, Västerås, Sweden*, 2023.